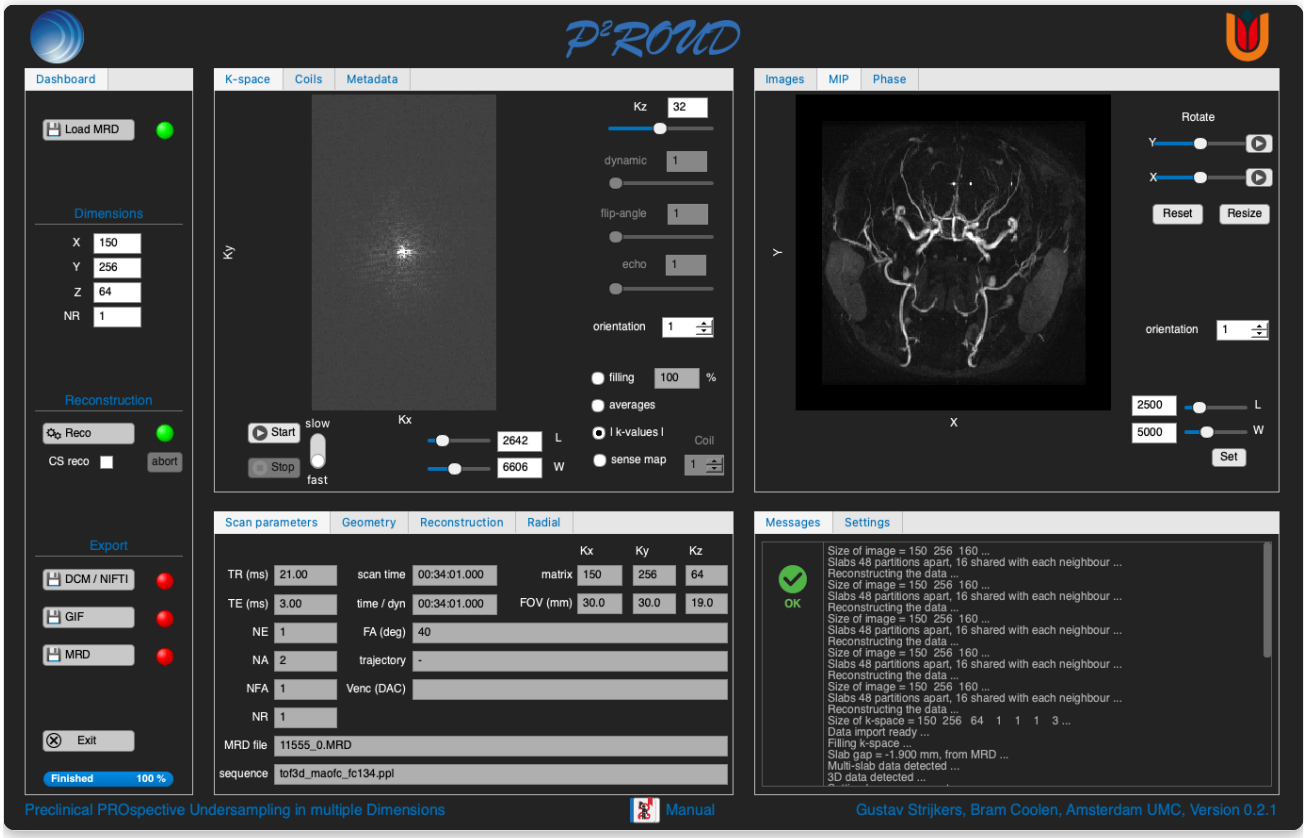


Table of contents

1. Overview
2. Contact
3. Disclaimer
4. Installation
5. Running the software
6. Basic operation
7. Credits

1. Overview

P2ROUD = Preclinical PROspective Undersampling in multiple Dimensions.

P2ROUD is an application with a graphical user interface to reconstruct MRI data acquired with an MR Solutions preclinical MRI system, in particular data with k-space undersampling in one or more dimensions. Besides Cartesian 2D and 3D data it reconstructs 2D radial, 3D UTE and ZTE scans, and 2D EPI scans for diffusion and fMRI, which it can export as a BIDS dataset.

P2ROUD is a stand-alone application: everything it uses is inside it, and nothing else has to be installed.

2. Contact

P2ROUD and this manual are written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.strijkers@amsterdamumc.nl

and

Bram Coolen
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
b.f.coolen@amsterdamumc.nl

3. Disclaimer

This is a beta version. The software has been extensively tested using mouse and rat data acquired with an MR Solutions preclinical 7.0T/24cm system. However, this does not warrant that the functions contained in the program will meet your requirements or that the operation of the program will be uninterrupted or error-free. In case of questions or issues, please contact Gustav Strijkers or Bram Coolen.

4. Installation

macOS

Requirements: a Mac with Apple silicon (M1 or later), macOS 26 or later.

P2ROUD comes as a disk image, `P2ROUD-0.2.10-macOS-arm64.dmg`. Nothing else has to be installed: Python, dcm2niix and everything else the app uses are inside it.

1. Open the disk image and drag **P2ROUD** onto the Applications folder in its window.
2. Open it from Applications. The app is not signed with an Apple Developer ID, so the first time macOS refuses it ("Apple could not verify ..."); click **Done**. Then open **System Settings > Privacy & Security**, scroll down to the line saying P2ROUD was blocked, click **Open Anyway** and confirm. From then on it opens as any other app.

   Instead of step 2, the same in Terminal, once:

   ```
   xattr -dr com.apple.quarantine /Applications/P2ROUD.app
   ```

The first start can take a little longer: macOS checks the app once.

Windows

Requirements: a PC with Windows 10 or 11, 64-bit.

P2ROUD comes as an installer, `P2ROUD-0.2.10-setup.exe`. Nothing else has to be installed: Python, dcm2niix and everything else the app uses are inside it.

1. Run the installer. It is not signed, so the first time Windows may stop it ("Windows protected your PC"): click **More info** and then **Run anyway**.
2. The installer puts the app in `%LOCALAPPDATA%\Programs\P2ROUD` for the current user, without administrator rights (installing it for all users is an option), with a Start-menu entry and, if you choose, a desktop shortcut. At the end it can start the app.

To remove the app, use **Uninstall P2ROUD** in the Start menu, or **Settings > Apps**.

BART (optional)

With BART, the Berkeley Advanced Reconstruction Toolbox (https://mrirecon.github.io/bart/), the compressed sensing reconstructions run with BART. Without it, the app reconstructs with its own engine, based on sigpy, with the same settings; it takes two to six times as long. P2ROUD is tested with BART 1.0.00.

At its start the app writes BART's version in the message box, followed by "BART toolbox detected ..." or "WARNING: BART toolbox not available ...", and whether BART was compiled with GPU support. A BART built for the GPU is used on the GPU.

Installing BART on macOS with MacPorts

1. Install the Xcode command line tools:

   ```
   xcode-select --install
   ```

2. Install MacPorts from https://www.macports.org/.
3. Install the packages:

```
sudo port install fftw-3-single gcc14 libpng openblas flock gmake coreutils l
ibblocksruntime
```

4. Download and unpack BART:

```
wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
tar xvfz v1.0.00.tar.gz
cd bart-1.0.00
```

5. Build and install:

```
export PATH=/opt/local/libexec/gnubin:$PATH
CC=gcc-mp-14 gmake
sudo gmake install
```

   The compiler name must match the gcc version installed in step 3: gcc14 gives gcc-mp-14, gcc15 gives gcc-mp-15.
6. Check that it works: `bart version`

Installing BART on macOS with Homebrew

1. Install the Xcode command line tools: `xcode-select --install`
2. Install Homebrew:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/instal
l/HEAD/install.sh)"
```

3. Install the packages BART needs:

```
brew install gcc libpng fftw openblas make llvm libomp
```

4. Download and unpack BART as above (step 4 of MacPorts).
5. Build and install:

```
CC=gcc-15 MACPORTS=0 gmake
sudo gmake install
```

   The compiler name must match the version Homebrew installed; check it with `ls /opt/homebrew/bin/gcc-*`.
6. Check that it works: `bart version`

How P2ROUD finds BART on macOS

In the normal case there is nothing to configure: `sudo make install` puts BART in `/usr/local/bin`, which is where the app looks first. It looks in these folders, in this order:

Folder	
`/usr/local/bin`	where BART's own `make install` puts it
`/usr/bin`	
`/opt/homebrew/bin`	Homebrew's
`/opt/local/bin`	MacPorts'

An app opened from the Finder or the Dock does not see the settings of your shell (such as `TOOLBOX_PATH` or `PATH` in your shell profile). If your `bart` is in another folder, a link to it in `/usr/local/bin` lets the app find it. In Terminal, once, with the folder your bart is in instead of `/path/to/bart` (Terminal asks for your password):

```
sudo mkdir -p /usr/local/bin
sudo ln -s /path/to/bart/bart /usr/local/bin/bart
```

Installing BART on Windows 11

On Windows, BART runs inside the Windows Subsystem for Linux (WSL), in the default distribution; the app calls it as `wsl bart`.

1. Install WSL. Start a Windows PowerShell as administrator and run:

   ```
   wsl --install
   ```

   This enables the feature, installs WSL 2 and installs Ubuntu in one step. A restart is needed. The first time you start the Linux prompt you are asked to create a user account. On older systems, or if `wsl --install` is not recognized, see https://pureinfotech.com/install-wsl-windows-11/.

2. Update the Linux distribution (at the Linux prompt):

   ```
   sudo apt-get update
   sudo apt-get dist-upgrade
   ```

3. Install the packages BART needs (gcc 12 or newer):

   ```
   sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev libopenb
   las-dev gfortran
   ```

4. Download and unpack BART:

   ```
   wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
   tar xvfz v1.0.00.tar.gz
   cd bart-1.0.00
   ```

5. Build and install, or with GPU support as below:

   ```
   make
   sudo make install
   ```

6. Check that it works: at the Linux prompt `bart version`, and from a Windows command prompt `wsl bart version`.

Building BART with GPU support on Windows (provisional, highly recommended)

1. Install the NVIDIA driver in Windows, as described at https://docs.nvidia.com/cuda/wsl-user-guide/index.html. It is what gives WSL the GPU; no Linux display driver goes inside WSL.

2. Install the CUDA toolkit inside WSL, from https://developer.nvidia.com/cuda-downloads (Linux, x86_64, WSL-Ubuntu). For CUDA 12.8:

   ```
   wget https://developer.download.nvidia.com/compute/cuda/12.8.0/local_installe
   rs/cuda_12.8.0_570.86.10_linux.run
   sudo sh cuda_12.8.0_570.86.10_linux.run
   ```

3. Set the environment variables, in the shell you build in: `CUDA_BASE` for the CUDA version installed, `GPUARCH_FLAGS` for the card. BART has to be built for the card's generation (look it up at https://developer.nvidia.com/cuda-gpus): a build for another one compiles without complaint and fails at the first calculation.

Generation	Cards	GPUARCH_FLAGS
Pascal	GTX 1060, 1080, Titan X	`-arch=sm_61`
Turing	RTX 2060 to 2080	`-arch=sm_75`
Ampere	RTX 3060 to 3090	`-arch=sm_86`
	A100	`-arch=sm_80`
Ada Lovelace	RTX 4060 to 4090	`-arch=sm_89`
Blackwell	RTX 5070 to 5090	`-arch=sm_120`

```
export CUDA=1
export CUDA_BASE=/usr/local/cuda-12.8
export CUDA_LIB=lib64
export GPUARCH_FLAGS="-arch=sm_61"
export PATH=$CUDA_BASE/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_BASE/$CUDA_LIB:$LD_LIBRARY_PATH
```

4. Make. If the linking fails with an error about libstdc++ or libgcc, point the linker at the installed gcc with the last line (its number from `ls /usr/lib/gcc/x86_64-linux-gnu/`) and make again.

```
make -j$(nproc) CUDA=1 CUDA_BASE=$CUDA_BASE CUDA_LIB=$CUDA_LIB GPUARCH_FLAGS
="$GPUARCH_FLAGS"
sudo make install
export LDFLAGS="-L/usr/lib/gcc/x86_64-linux-gnu/13 $LDFLAGS"
```

5. Test: `make utest_gpu`

5. Running the software

Open **P2ROUD** from the Applications folder (macOS), or from the Start menu or the desktop shortcut (Windows). The P2ROUD logo appears while the app loads, and the window follows within seconds. At its start the app looks for BART, and the message box reports what it found; "Ready" says it is done.

Newer versions. At its start the app asks GitHub (https://github.com/Moby1971/MRI-apps) whether there is a newer release of it. The message box then shows the current and the newest version and, when there is a newer one, where to download it. Without a network connection it says that the version could not be checked, and the app works as usual. `checkForUpdates` in the settings file switches the check off; `versionCheckTimeout` sets how many seconds it waits for GitHub (5).

This manual. The **Manual** button at the bottom of the window, with the app's icon on a book, opens this manual in the computer's PDF viewer.

The window has a fixed size. On a screen too small for it, the window fits the screen and can be scrolled.

The first Load MRD dialog opens in the scanner's MRD folder (on the scanner computer, Windows), or in the folder written in `mrdpath.txt`, a text file in the settings folder below.

Settings and log files

P2ROUD keeps its files in a folder of its own, which it makes at its first start:

macOS	`~/Library/Application Support/P2ROUD/0.2.10p`
Windows	`%APPDATA%\P2ROUD\0.2.10p`

Settings. The settings are kept in `p2roudSettings.json`. The file is written with the standard values the first time a version starts, and can then be edited by hand; changes take effect at the next start. It holds, among others, the default compressed sensing weights and iterations for each type of data, whether the EPI corrections and the ZTE registration are ticked at start, the 3D UTE trajectory file chosen in the Radial tab, and the field strength written into DICOM headers made without the scanner's own when the scan's `.rpr` file gives none (7 T). A value that cannot be used is replaced by its standard value, and the message box says so.

Log files. Every session writes a log file, `p2roud-log-<date and time>.txt`, to the same folder. The log keeps every message, also those the message box does not show at the chosen message level. The newest 50 log files are kept; older ones are removed when the app starts. The number is `logFilesToKeep` in `p2roudSettings.json`. No other file in the folder is touched.

Other files in the folder. `ktraj.txt`, the app's 3D UTE readout profile and the default trajectory file (see step 1), written at the start when it is not there; `3DUTEgradDelays.txt`, the six gradient delays a 3D UTE scan loads with (see *Reconstruction of radial scans*), written with zeros at the start when it is not there; `mrdpath.txt`, when you add it; and `graddelays.txt`, when you add one, which takes the place of the app's own gradient delays.

6. Basic operation



The P2ROUD program operates from a single window with 5 panels.

Panel 1: Dashboard. The task buttons and parameters that control the reconstruction process. The dashboard tasks need to be completed from top to bottom. A green light next to a task indicates that the task has been completed; yellow, that it is busy; red, that it has not been completed yet, or failed. At the bottom of the Dashboard one progress bar shows the progress of every task: what it is doing and how far it has got ("Loading", "Reconstruction", "Export 30 of 120", "Finished").

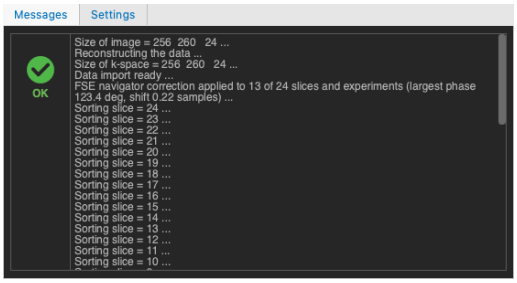
Panel 2: K-space / Coils / Metadata. The raw k-space data, and the coils and their sensitivity maps. The **Metadata** tab beside them holds the record of the scan: the animal, the session and the contrast agent (step 8).

Panel 3: Scan parameters / Geometry / Reconstruction / Radial. The acquisition, geometry, reconstruction and radial reconstruction parameters.

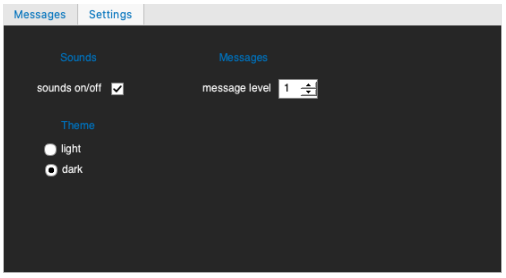
Panel 4: Images / MIP / Phase. After reconstruction, the reconstructed images.

Panel 5: Messages / Settings. Program status and messages, newest on top. In the Settings tab the sounds can be switched on or off (**sounds on/off**), the **light** or **dark** theme chosen (dark is the default), and the **message level** (1 to 3) set: how much the message box shows. The status icon shows OK, WARNING, ERROR or BUSY.

Messages



Settings



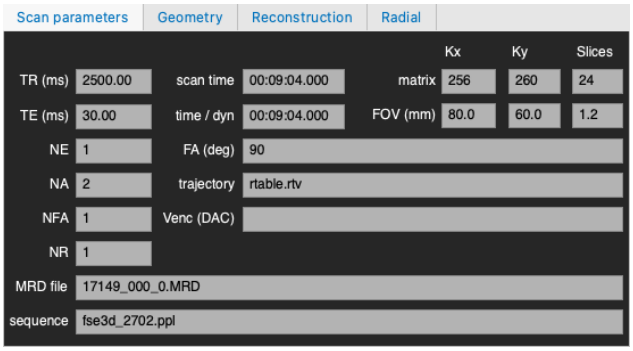
Step 1: Loading data

Press **Load MRD** to load the MRD raw data file.

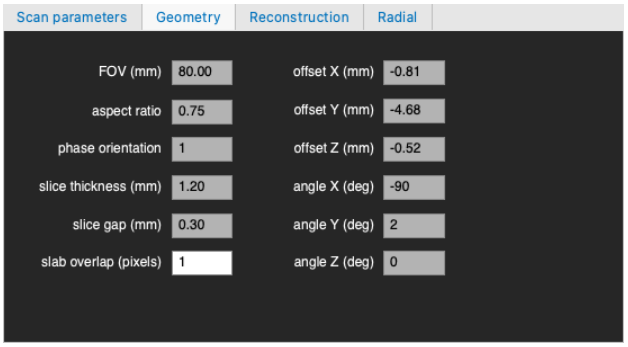
P2ROUD can reconstruct spin-echo, fast spin-echo, gradient-echo, MPRAGE and CINE data, multi-slab 3D, 2D radial, 3D UTE and ZTE data, and 2D EPI (diffusion and fMRI). However, the program is particularly designed to reconstruct undersampled data with a user-defined undersampling pattern (e.g. SENSE, random undersampling, pseudo-spiral or radial).

For data acquired with multiple receivers (phased-array RF coil) select one of the MRD files; the app finds the others. The relevant acquisition parameters are shown in panel 3.

Scan parameters



Geometry



k-space trajectory. There are three options to include a user-defined k-space trajectory in the FLASH (version 635e) sequence.

1. **2D data** — pe1-order = 3 ("LUT"). The k-space points for the 1st phase-encoding direction (views) are provided in the views table in the sequence. The trajectory field shows `mul_kspace_traj`.
2. **3D data** — pe1-order = 4 ("exLUT"). The k-space points for the 1st and 2nd phase-encoding directions (views and views2) are provided in a text file (`*.txt`) as a list of points without comma separation, for example:

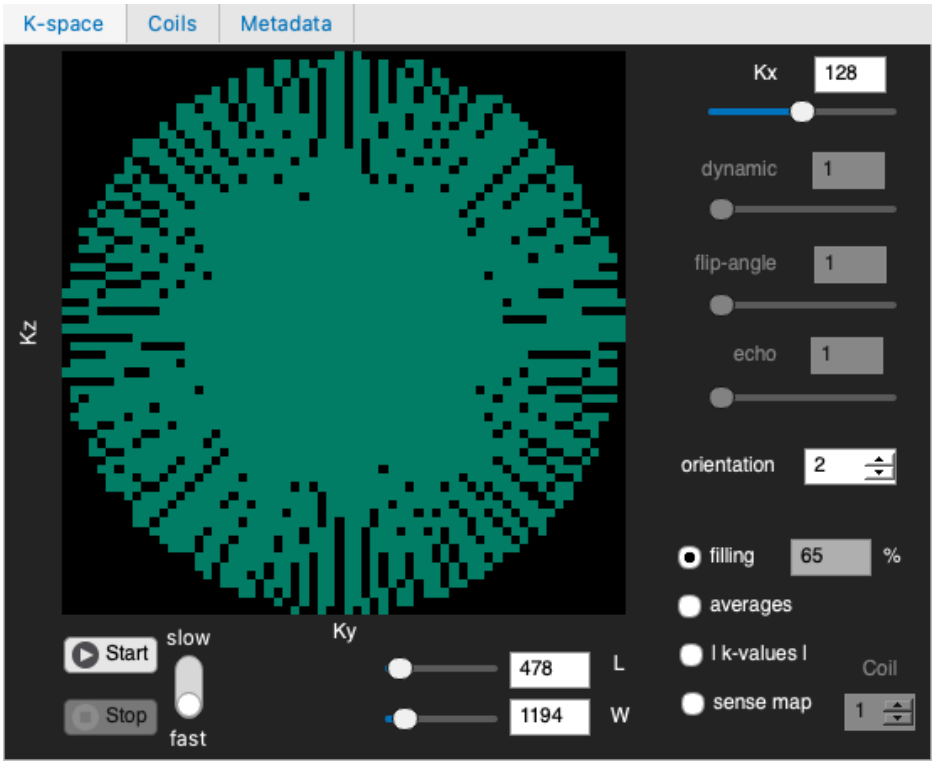
```
-48       (views   k-point 1)
0         (views2  k-point 1)
-47       (views   k-point 2)
0         (views2  k-point 2)
-46
1
...
```

3. **Undersampled 2D and 3D data** — pe1-order = 5 ("nrLUT"). The trajectories are read from the first `.txt` file in the folder of the MRD file (3D UTE: `lut*.txt`).

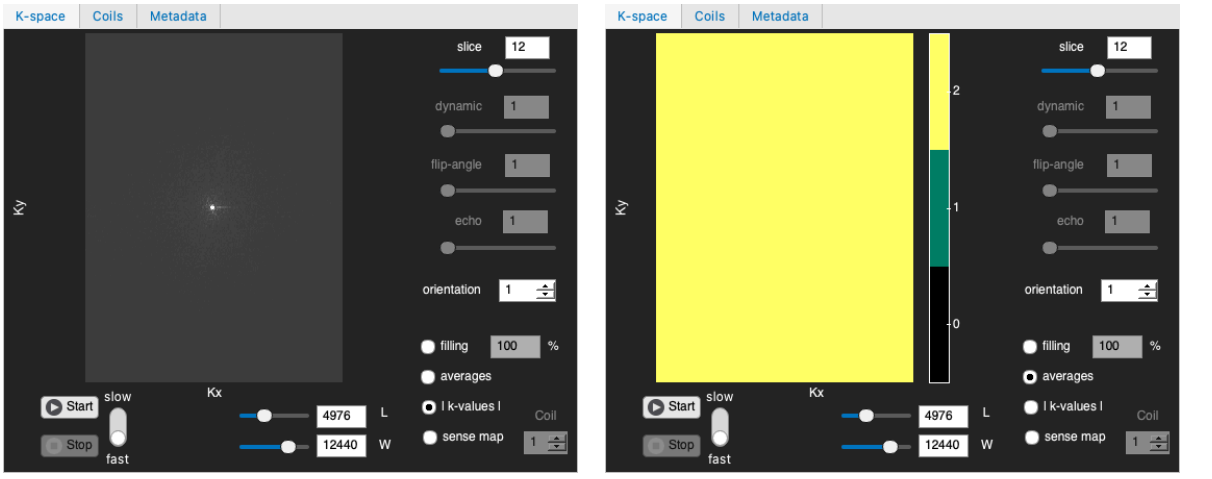
The readout profile of 3D UTE data is read from the trajectory file chosen in the UTE section of the Radial tab (step 6), by default `ktraj.txt` in the settings folder, which the app writes there at its start when it is not there. The file has one column for all axes or three for x, y and z. A comment line in it gives the sample period the profile was made for (`% sample period 2.5 us`; a file without one is taken as 2.5 µs); a scan with another sample period gets the profile converted to its own, which the message box says. A comment line can also give the data offset (for example `% data offset 7.5`), which is then used instead of the one in the Radial tab, for scans with the file's sample period.

Step 2: Inspecting k-space

The raw k-space data is displayed in the K-space tab of panel 2. The radio buttons show the acquired k-space points (**filling**), the averages per k-point (**averages**), the absolute value of k-space (**| k-values |**) or a coil sensitivity map (**sense map**). The percentage of k-space points acquired is shown beside **filling** (65 % for the dataset below).



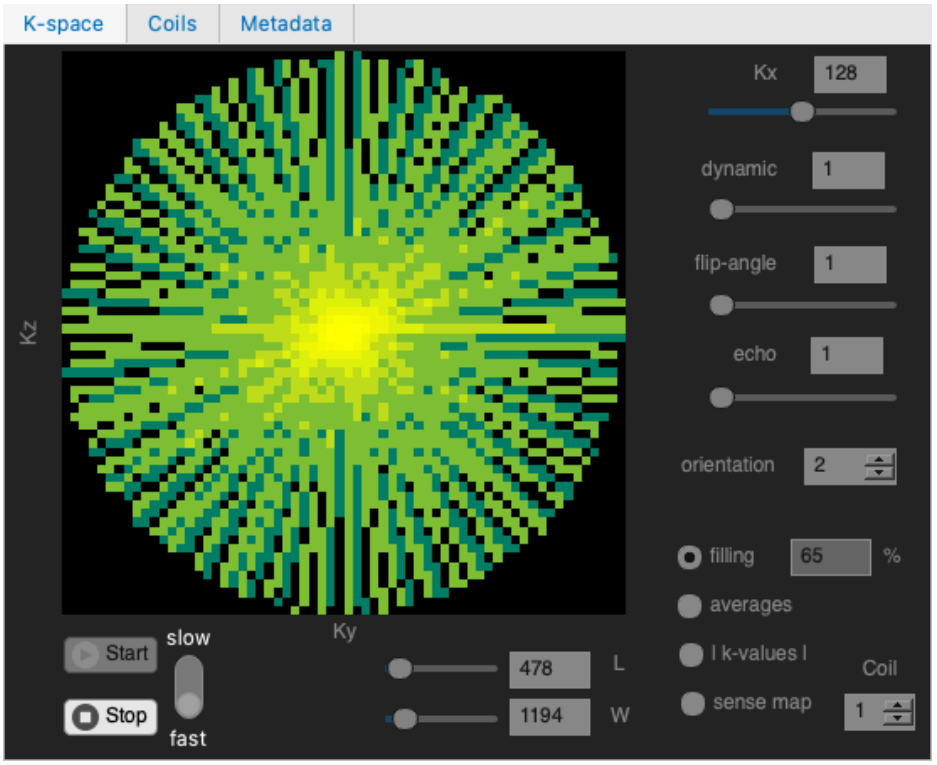
| k-values | **averages**



The sliders step through the third k-space dimension (**Kz**, or **slice** for 2D data), the repetitions (**dynamic**), the flip angles (**flip-angle**) and the echoes (**echo**), if applicable; for a series that steps through TR, TI or CINE frames the slider is named so. The **orientation** spinner chooses the k-space plane shown, the **Coil** spinner the coil, and **L** and **W** set the level and window.

Step 3: The k-space trajectory

Press **Start** to start an animation of the k-space trajectory, and **Stop** to stop it. The switch beside them toggles between **fast** and **slow** animation.



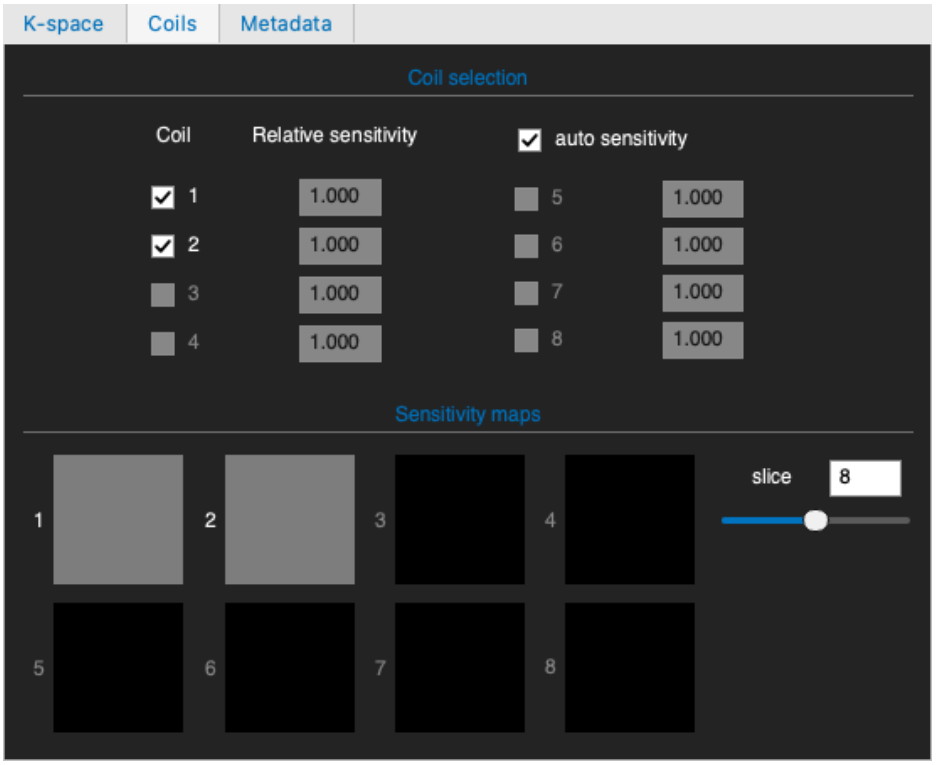
Step 4: Fourier transform reconstruction

For a first quick inspection of the reconstructed images, press **Reco** with **CS reco** unticked: a Fourier transform reconstruction.

X, **Y** and **Z** in the Dimensions panel set the size of the images: for Cartesian data k-space is zero-filled or cropped, for radial, UTE and ZTE data the images are resized. For 2D data the third field is **Slices** and keeps the number of slices acquired. **NR** sets the number of repetitions (see step 6).

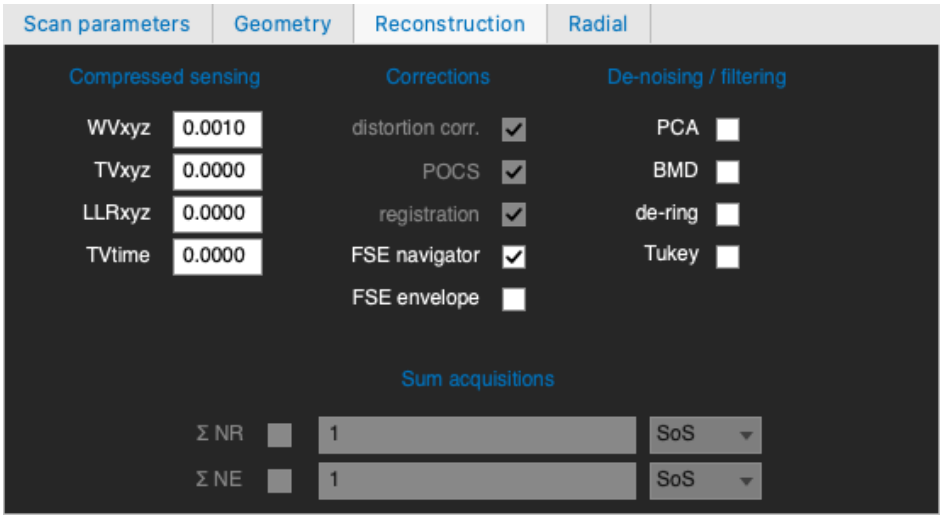
Step 5: Coils

For acquisitions with a phased-array coil the Coils tab gives access to the channels: coils can be checked or unchecked to use or omit them in the reconstruction. Relative sensitivities can be set by hand, or determined with **auto sensitivity**, which for a CS reconstruction also computes the ESPIRiT coil maps shown under **Sensitivity maps**. Auto sensitivity is ticked at every load of multi-coil data.



Step 6: Reconstruction

Ticking **CS reco** in the Dashboard gives access to the Compressed Sensing (CS) reconstruction parameters in the Reconstruction tab of panel 3.



The CS regularization parameters are:

1. **WVxyz** — wavelet regularization in the spatial (x, y and z) dimensions.
2. **TVxyz** — total variation constraint in the spatial dimensions.
3. **LLRxyz** — locally low-rank constraint in the spatial dimensions.
4. **TVtime** — total variation constraint in the dynamics (repetitions, NR) dimension.

The spatial dimensions of the reconstructed data can be increased or decreased. The number of repetitions (**NR**) can also be changed to undersample or average data in the dynamics dimension.

Pressing **Reco** starts the CS reconstruction, with BART when the app found it (section 4). The number of iterations is fixed for each type of data; the settings file holds it. A running reconstruction can be stopped with **abort** in the Dashboard.

For multi-slab 3D (MOTSA) scans, the slabs are put where they were acquired and blended over **slab overlap (pixels)**, in the Geometry tab, on either side of each junction.

Further options in the Reconstruction tab

De-noising / filtering is applied to the reconstructed images; the methods can be combined.

- **PCA** removes noise by principal component analysis over the images of a series (dynamics, diffusion directions, flip angles, echoes).
- **BMD** applies block-matching denoising.
- **de-ring** suppresses Gibbs ringing. It is not available for radial, UTE and ZTE data.

Corrections.

- **Tukey** tapers the edges of k-space against ringing, at some cost in resolution; loading a Cartesian scan unticks it, a radial or UTE scan ticks it.
- **FSE navigator** (ticked by default) corrects the echoes of a fast spin echo scan with its navigator.
- **FSE envelope** reduces the ghosting of a fast spin echo scan at the same signal-to-noise ratio; the images are slightly less sharp.

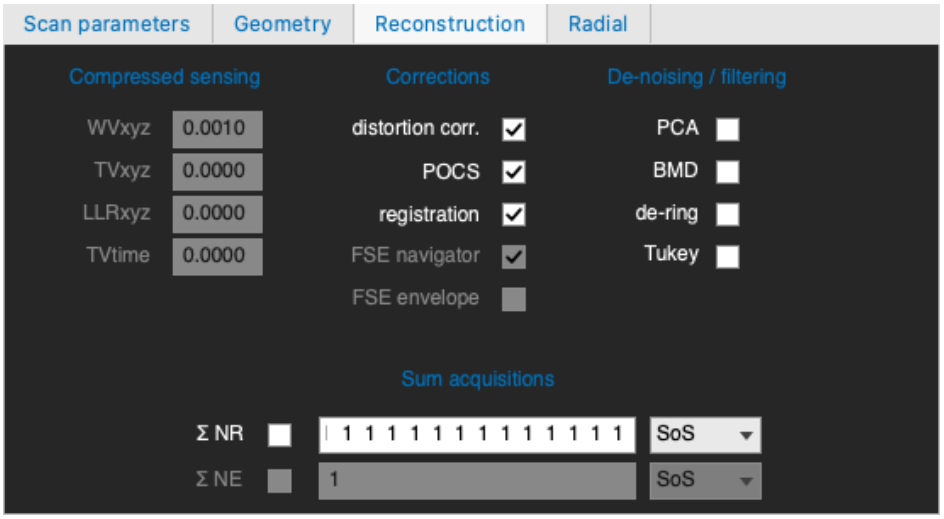
The FSE boxes are available for fast spin echo scans with a navigator.

Sum acquisitions. With **Σ NR** or **Σ NE** ticked, the dynamics or the echoes are combined into one image by **SoS**, **Max** or **Sum**; the field beside the box selects which, with a 1 or 0 for each.

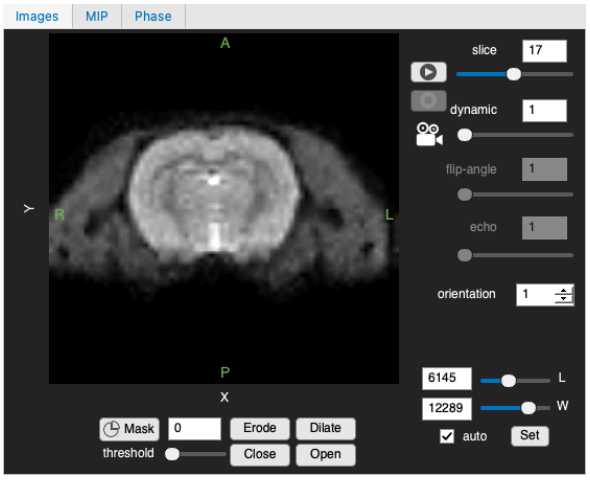
EPI scans

2D EPI scans, single-shot or read-interleaved, spin echo or gradient echo, with or without partial Fourier, are reconstructed as diffusion series (DTI, several b-values) or fMRI time series. The images are masked when the study has a mask scan of the same geometry (named in the scan settings, or the nearest one). Three corrections in the EPI part of the tab are on by default:

- **distortion corr.** corrects the geometric distortion with a partner scan of the same study, with the same slices and echo train and the opposite phase-encoding direction, which the app finds by itself. Without a partner scan the message box says so and nothing is changed.
- **POCS** completes a partial Fourier scan: sharper images, at some cost in signal-to-noise ratio.
- **registration** aligns the volumes of the series in the image plane.



Diffusion (DTI, 130 volumes): a b = 0 volume



fMRI (600 dynamics): slice 6, dynamic 300

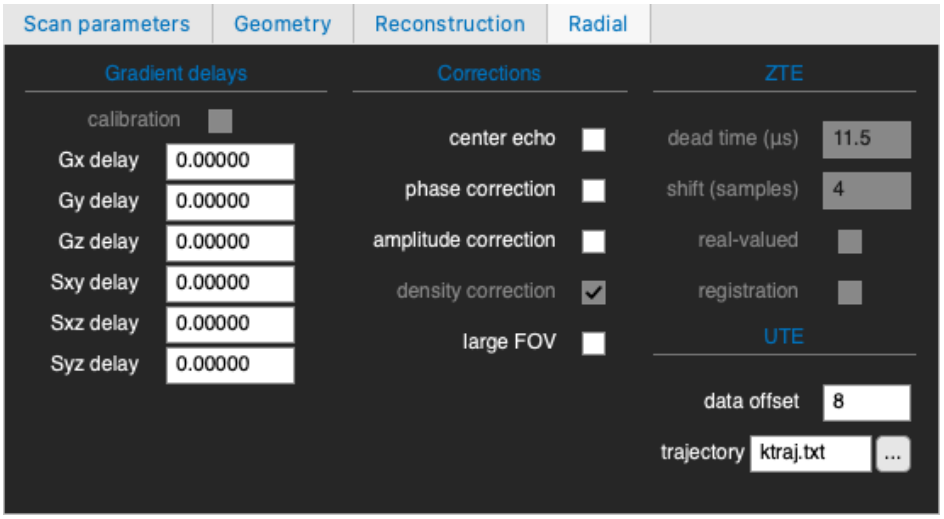


The **DCM / NIFTI** export of an EPI series also writes a BIDS dataset in `BIDS/sub-<study>/ses-1`: `func` (fMRI) or `dwi` with the gradient table (diffusion), and `anat`, the fast spin echo scans of the study (T1w and/or T2w), which the app finds and reconstructs itself. An fMRI series needs such a scan, a diffusion series does not.

For a diffusion scan with at least 6 directions and a b = 0 volume, MRtrix3 and DSI Studio scripts for tractography (`tractography_MRtrix_<tag>` and `tractography_DSIStudio_<tag>`, `.sh` and `.bat`) are written into the NIfTI folder; the app does not run them.

Reconstruction of radial scans

The Radial tab gives access to advanced reconstruction parameters for 2D radial, 3D UTE and ZTE scans.

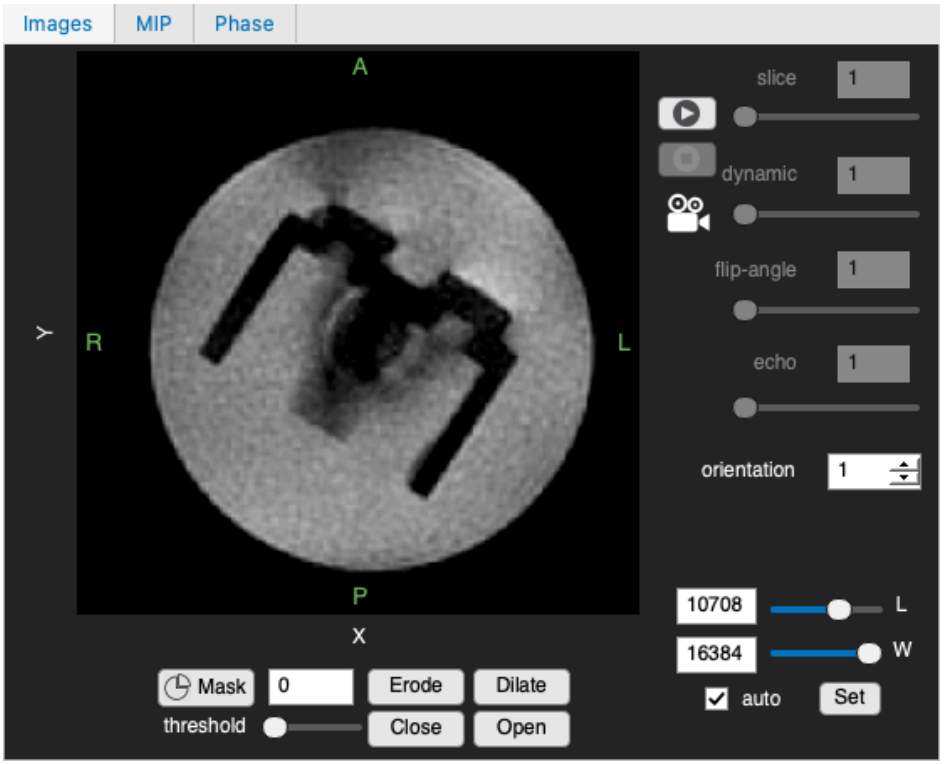


Tick **calibration** to estimate the gradient delays of a 2D radial scan from the data at the next reconstruction; the delay fields are then filled in and the checkbox is unticked. 2D radial scans load with calibration ticked; the calibration takes a few seconds, and **center echo** does not apply. **phase correction** corrects the phase of every spoke. For 3D UTE and ZTE scans the delays are entered in the fields. The delays, in samples, are **Gx**, **Gy** and **Sxy** for 2D radial data, and **Gx**, **Gy**, **Gz**, **Sxy**, **Sxz** and **Syz** for UTE and ZTE; they are reset when a data set is loaded. A 3D UTE scan loads with the six delays of `3DUTEgradDelays.txt` in the settings folder: six numbers in that order, all 0 as the app writes the file. Put in your own values to have every 3D UTE scan start with them; lines starting with `%` are comments. A file that cannot be read, or does not hold six numbers, is replaced by a new one with zeros, and the message box says so.

The **UTE** section applies to 3D UTE scans only; for other scans it is grayed out. **data offset** is the number of samples at the start of each readout before the trajectory starts; these are left out. It may have decimals: with 7.5 the first 7 samples are left out and the trajectory is moved half a sample along the readout, so the result lies between those of 7 and 8. An even number of samples is kept for the reconstruction, so with an odd count the last sample of the readout is left out too. **trajectory** names the file with the readout profile, `ktraj.txt` by default. Type a name for a file in the settings folder, or click **...** to choose a file; a file outside the settings folder is kept with its full path. The choice is kept in the settings file for the next start, and a loaded 3D UTE scan uses the new file at once. When the file is missing or cannot be used, the message box says so and the app's own profile is used.

2D radial images are black outside the circle the spokes cover, in the corners of the image.

The Images tab below shows a reconstructed 2D radial scan of a phantom.



For 3D UTE, **center echo** centers the echoes before reconstruction and **phase correction** corrects their phase; a 3D UTE scan loads with both unticked and data offset 6 (sample period 2.5 µs) or 0.6 (5 µs).

large FOV lets less signal leak into a 3D UTE image from beyond the field of view; it takes about a quarter longer. It applies to the NUFFT reconstruction, not to CS.

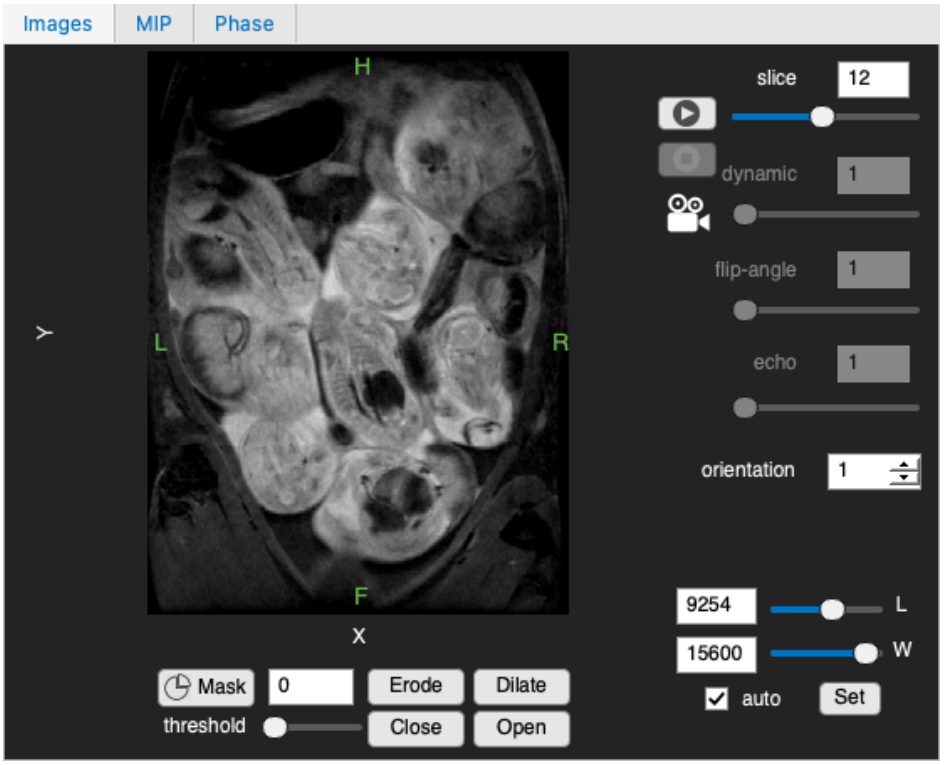
density correction performs a 2D or 3D density correction in k-space to account for non-uniform sampling. It only applies to CS reconstructions, and its checkbox is on only with **CS** ticked.

amplitude correction evens out the signal amplitude of the spokes. The Tukey filter, which tapers the readout, is set in the Reconstruction tab.

ZTE scans have no calibration and none of these corrections. Their **dead time (µs)** is the delay between the excitation and the first sample, 11.5 µs by default, and **shift (samples)** the first sample kept, 4 by default; changing shift rebuilds k-space. The reconstruction fills in the samples before the first one kept: too small a dead time leaves solid objects dark inside with bright edges, too large a value makes them bulge in the middle. **real-valued** gives a darker background while the object keeps its signal. **registration** aligns the dynamics of a series in 3D.

Step 7: Viewing the reconstructed images

After reconstruction the MR images are displayed in panel 4.



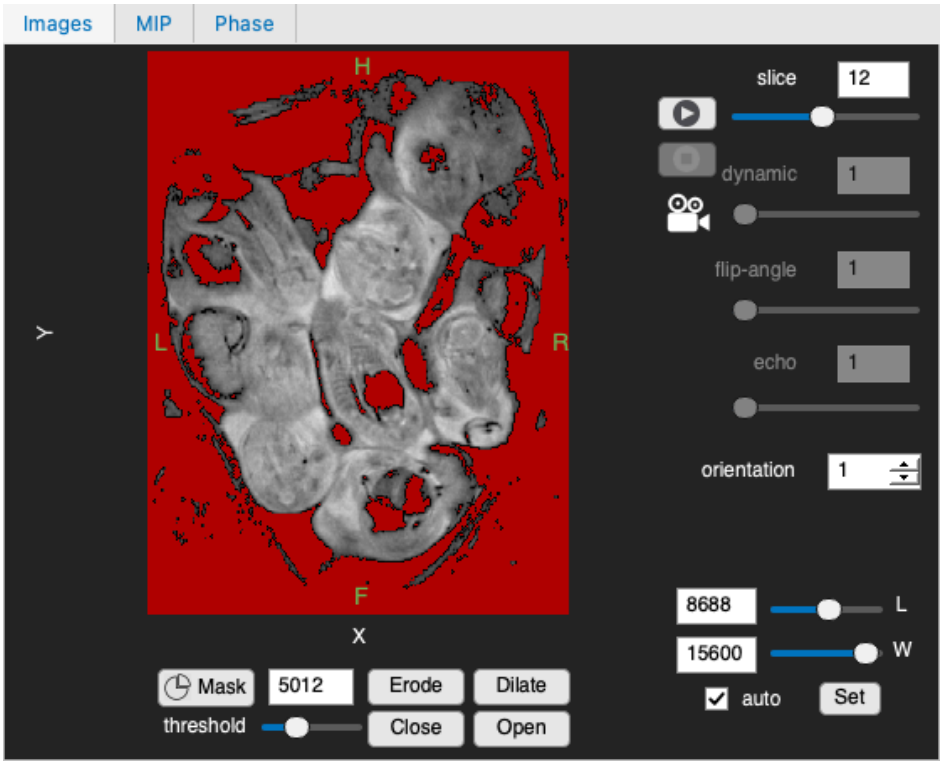
The image orientation can be changed with the **orientation** spinner; the green letters around the image give the anatomical directions. The MIP and Phase tabs follow it.

Image intensity can be adjusted with the level (**L**) and window (**W**) sliders. Checking **auto** adjusts the intensity automatically; **Set** takes the level and window of the image shown and unticks auto.

Press the play button to start a movie of the images as a function of the dimension shown next to the camera icon (for example the dynamics), and the stop button to stop it.

When the pointer is over the image, a small toolbar appears: its zoom button zooms with a box, a click or the wheel, and its home button (**Whole image**) shows the whole image again. With zoom off, the scroll wheel steps through the slices.

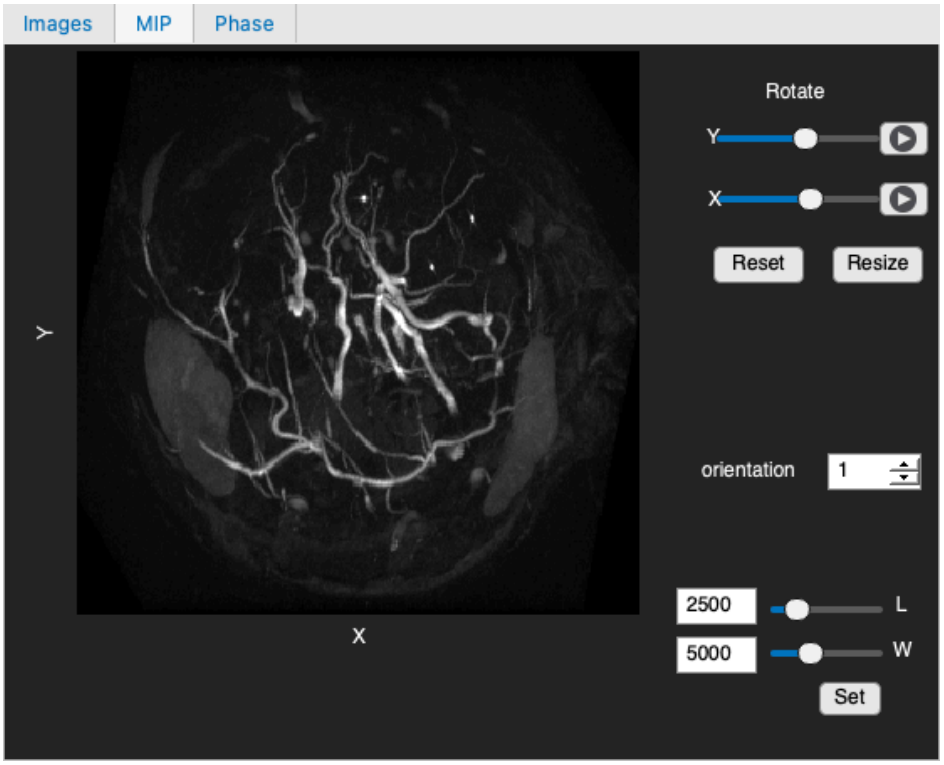
Images can be segmented (on the basis of a threshold) by pressing **Mask**. Use the **threshold** slider or the morphological operations (**Erode**, **Dilate**, **Open**, **Close**) to fine-tune the segmentation. The masked-out pixels are shown in red. The mask is used for display, the MIP and phase unwrapping only.



Maximum intensity projection (MIP)

In the MIP tab a maximum intensity projection is shown. The MIP can be turned with the **X** and **Y** sliders under **Rotate**, and animated with the play buttons beside them, 5° per step (pressing again stops). **Reset** stops the movies and returns to the start; **Resize** halves the volume for speed. The MIP has its own level, window and **Set**.

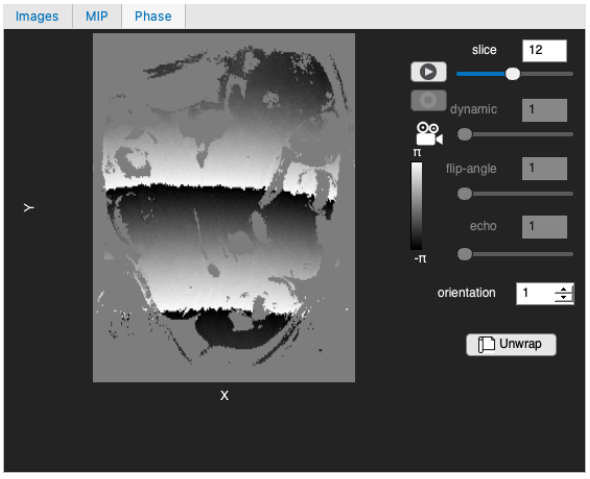
The object keeps its size as it turns: the view is the same at every angle, and the corners of a turned projection that reach beyond it are cut off.



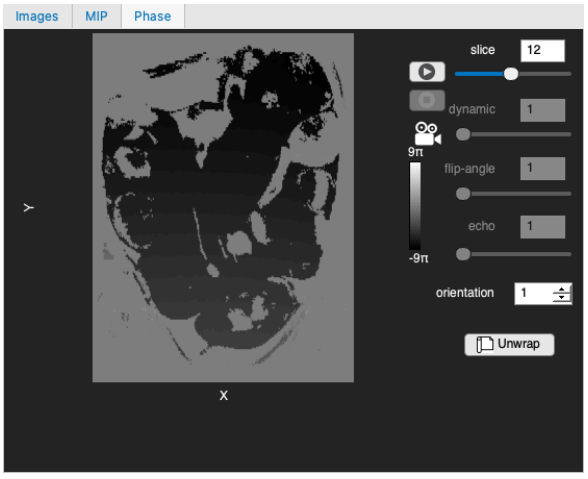
Phase images

The Phase tab shows the phase images, with their own sliders, movie and orientation. The phase can be unwrapped by pressing the **Unwrap** button; pixels outside the mask keep their wrapped value.

Phase



Unwrapped

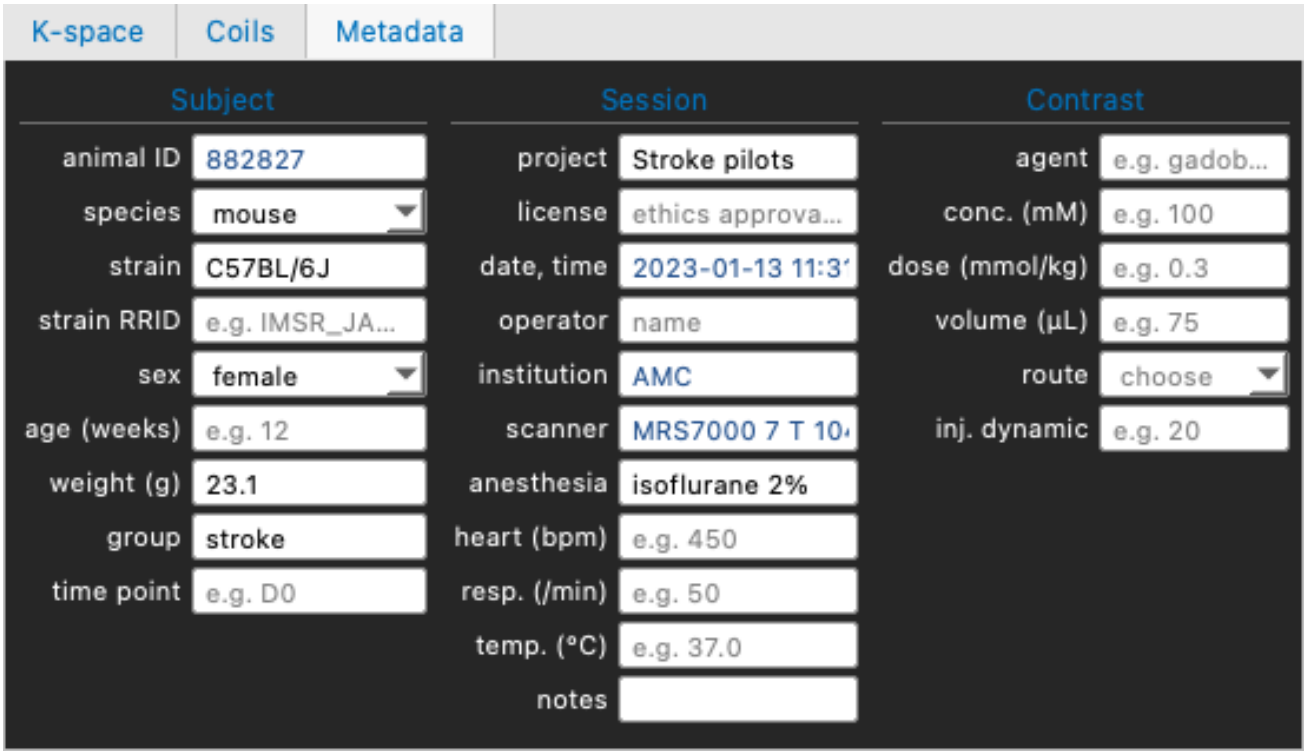


Step 8: Metadata

The **Metadata** tab, beside the K-space and Coils tabs, holds the record of the scan, which the DCM / NIFTI and GIF exports write with the images. Its fields stand in three columns:

- **Subject:** animal ID, species, strain and its RRID, sex, age, weight, group and time point;
- **Session:** project, ethics license, date and time, operator, institution, scanner, anesthesia, heart and respiration rate, and temperature, with a field for notes;

- **Contrast:** agent, concentration, dose, volume, route and the dynamic it was injected at.



At a load the program fills in what the scan's files say: from the `.rpr` file beside the MRD file the animal ID and the time point (from the patient name typed at the console: `20240212_926752_D0` is animal 926752, day 0), the date and time, the institution and the scanner, and from the scanner's DICOM images, where they are found, the project. What the console leaves at its standard values (sex O, weight 0, the login as the operator) is left empty.

What was filled in for an earlier scan of the same exam is filled in again, from the records the apps wrote in the exam's `Metadata` folder (P2ROUD, Retrospective and the mapping apps alike) and from the tab as it was for the scan loaded before: the animal's details are typed only once per exam. The date, the heart and respiration rate, the temperature and the notes belong to one scan and are never taken from another.

A value the program filled in is shown in blue; every field can be corrected, and then counts as yours. Species, sex and route are lists to choose from, or to type in; numbers are kept with their units (a number field holding something else is edged in red). The species and sex are written with their codes (NCBI Taxonomy, PATO), so the record can be read by other software as well as by people.

Step 9: Image export

There are three ways to export the images, in the Export panel of the Dashboard.

1. **DCM / NIFTI** — exports the images as DICOM files for further processing in third-party software. When the scanner's own DICOM images are found, their headers are used; otherwise the headers are made from the MRD data, with the orientation taken from the scan angles and the slices centered; the patient, institution and scanner in them are those of the scan's `.rpr` file, and stay empty where it has none. The DICOM files carry the species, strain, sex, age and weight of the animal where the Metadata tab has them. The images are also written as NIfTI files with their

geometry (with dcm2niix), and for diffusion scans with `.bval` and `.bvec` files; `recoparameters_<tag>.txt` records the scan and the settings used. For EPI series a BIDS dataset and scripts for tractography are written as well (see "EPI scans"). When the scanner's DICOM export of the scan is found in …`/Data/<study>/DICOM/<scan>`, the files go into …`/Data/<study>/DICOM/<series>P/<n>` and `Nifti/<tag>P/<n>`, a new numbered folder `<n>` each time; otherwise into a folder that is asked for (`DICOM/<tag>P/<n>` there).

2. **GIF** — the data are exported as (animated) GIF files, one per slice, flip angle or echo. A zoomed image is exported as far as it is in view. When the MRD file lies in …`/MRD/<study>/<scan>`, they go into …`/Data/<study>/GIF/<tag>P/<n>`, a new numbered folder each time; otherwise into a folder that is asked for.
3. **MRD** — the data are stored as a new MRD file, `p2roud_<n>_<tag>.MRD`, in the folder of the input MRD file (or in a folder asked for, when that cannot be written), with `recoparameters_<tag>.txt` and, if the program finds an input `.rpr` file, a new `.rpr` file. The scan's `rtable.rtv` is renamed `origrtable.rtv.orig`. Additionally, when the program runs on the scanner computer (Windows), the data are reconstructed by the scanner software and new SUR files, and for nrLUT scans DICOM images made from them, are placed in the data directory for viewing and planning of next scans in the MR Solutions preclinical software. nrLUT scans are exported to MRD by themselves after every reconstruction.

The DCM / NIFTI and GIF exports also write, in …`/Data/<study>/Metadata/<tag>P/<n>` (or under the folder asked for):

- `metadata.json` — the Metadata tab's record;
- `p2roud-report_<tag>.json` and `.txt` — the report of the export: the program and its libraries, the scan and a checksum of each of its files, the steps of the session, the scan and reconstruction settings, and every file the export wrote with its checksum;
- `ro-crate-metadata.json` — the same as an RO-Crate (https://www.researchobject.org/ro-crate/), a standard description of the folder for data repositories.

The paths in them are relative to the study folder, so they hold no names of your own folders.

Step 10: Exit

Press **Exit** to shut down the program. Closing the window does the same, also while the program is busy: the window disappears, and the program closes as soon as the running step has finished. Nothing the program started (BART) keeps running after it.

7. Credits

- **BART toolbox and compressed sensing algorithms** — https://mrirecon.github.io/bart/. Michael Lustig, David Donoho, John M. Pauly, Sparse MRI: The application of compressed sensing for rapid MR imaging, Magn Reson Med 58, 1182–1195 (2007).
- **sigpy** — Frank Ong and Michael Lustig, SigPy: a python package for high performance iterative reconstruction, Proc. Intl. Soc. Mag. Reson. Med. 27, 4819 (2019). Used for compressed sensing without BART.
- **Gibbs ringing suppression** — Kellner, E., Dhital, B., Kiselev, V.G. and Reisert, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med 76(5), 1574–1581 (2016).
- **NUFFT** — https://web.eecs.umich.edu/~fessler/
- **Phase unwrapping** — M. A. Herráez, D. R. Burton, M. J. Lalor, and M. A. Gdeisat, "Fast two-dimensional phase-unwrapping algorithm based on sorting by reliability following a noncontinuous path", Applied Optics, Vol. 41, Issue 35, pp. 7437–7444 (2002); H. Abdul-Rahman, M. Gdeisat, D. Burton, M. Lalor, "Fast three-dimensional phase-unwrapping algorithm based on sorting by reliability following a non-continuous path", Proc. SPIE 5856, Optical Measurement Systems for Industrial Inspection IV, 32 (2005).
- **PCA denoising** — J. Veraart, D. S. Novikov, D. Christiaens, B. Ades-Aron, J. Sijbers, E. Fieremans, Denoising of diffusion MRI using random matrix theory, NeuroImage 142, 394–406 (2016); L. Cordero-Grande, D. Christiaens, J. Hutter, A. N. Price, J. V. Hajnal, Complex diffusion-weighted image estimation via matrix recovery under general noise models, NeuroImage 200, 391–404 (2019).
- **Block-matching denoising**
  - Y. Mäkinen, L. Azzari, A. Foi, "Collaborative Filtering of Correlated Noise: Exact Transform-Domain Variance for Improved Shrinkage and Patch Matching", IEEE Transactions on Image Processing, vol. 29, pp. 8339–8354 (2020).
  - M. Maggioni, V. Katkovnik, K. Egiazarian, A. Foi, "Nonlocal Transform-Domain Filter for Volumetric Data Denoising and Reconstruction", IEEE Transactions on Image Processing, vol. 22, pp. 119–133 (2013).
  - Y. Mäkinen, S. Marchesini, A. Foi, "Ring Artifact and Poisson Noise Attenuation via Volumetric Multiscale Nonlocal Collaborative Filtering of Spatially Correlated Noise", Journal of Synchrotron Radiation, vol. 29, pp. 829–842 (2022).
- **NIfTI conversion** — dcm2niix: C. Rorden et al., https://github.com/rordenlab/dcm2niix.